EXPERIMENT 4

EXPLORATORY DATA ANALYSIS (EDA) – DATA INSPECTION, FILTERING, STATISTICAL ANALYSIS AND CORRELATION

Aim

To perform data inspection, filtering, grouping, aggregation, descriptive statistical analysis, and correlation analysis using Pandas and visualize the results using Matplotlib.

Algorithm 1

Dataset Inspection

Step 1

Import dataset

Step 2

Display dataset

Step 3

Check dimensions

Step 4

Display data types

Step 5

Generate summary

Program 1

```
In [1]: import pandas as pd
        import matplotlib.pyplot as plt

In [3]: df = pd.read_csv("healthcare-dataset-stroke-data.csv")

In [4]: df.head()
```

Out[4]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Program 2

Shape of Dataset

```
In [5]: print("Shape of Dataset:", df.shape)

        Shape of Dataset: (5110, 12)
```

Program 3

Data Types

```
In [6]: print(df.dtypes)

        id                   int64
        gender              object
        age                float64
        hypertension         int64
        heart_disease        int64
        ever_married        object
        work_type           object
        Residence_type      object
        avg_glucose_level  float64
        bmi                float64
        smoking_status      object
        stroke               int64
        dtype: object
```

Algorithm 2

Filtering Data

Step 1

Specify condition

Step 2

Apply filter

Step 3

Display records

Program 4

```
In [9]: age_above_60 = df[df["age"] > 60]
        age_above_60
```

Out[9]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1
5	56669	Male	81.0	0	0	Yes	Private	Urban	186.21	29.0	formerly smoked	1
...	...	...	...	...	...	...	...	...	...	...	...	...
5092	56799	Male	76.0	0	0	Yes	Govt_job	Urban	82.35	38.9	never smoked	0
5097	64520	Male	68.0	0	0	Yes	Self-employed	Urban	91.68	40.8	Unknown	0
5100	68398	Male	82.0	1	0	Yes	Self-employed	Rural	71.97	28.3	never smoked	0
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	NaN	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0

1304 rows × 12 columns

Program 5

```
In [10]: stroke_patients = df[df["stroke"] == 1]
         stroke_patients
```

Out[10]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1
...	...	...	...	...	...	...	...	...	...	...	...	...
244	17739	Male	57.0	0	0	Yes	Private	Rural	84.96	36.7	Unknown	1
245	49669	Female	14.0	0	0	No	children	Rural	57.93	30.9	Unknown	1
246	27153	Female	75.0	0	0	Yes	Self-employed	Rural	78.80	29.3	formerly smoked	1
247	34060	Male	71.0	1	0	Yes	Self-employed	Rural	87.80	NaN	Unknown	1
248	43424	Female	78.0	0	0	Yes	Private	Rural	78.81	19.6	Unknown	1

249 rows × 12 columns

Algorithm 3

Sorting Data

Step 1

Select column

Step 2

Apply sort_values()

Step 3

Display sorted result

Program 6

```
In [11]: sorted_bmi = df.sort_values(by="bmi", ascending=False)
         sorted_bmi.head()
```

Out[11]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
2128	56420	Male	17.0	1	0	No	Private	Rural	61.67	97.6	Unknown	0
4209	51856	Male	38.0	1	0	Yes	Private	Rural	56.90	92.0	never smoked	0
928	41097	Female	23.0	1	0	No	Private	Urban	70.03	78.0	smokes	0
544	545	Male	42.0	0	0	Yes	Private	Rural	210.48	71.9	never smoked	0
1559	37759	Female	53.0	0	0	Yes	Private	Rural	72.63	66.8	Unknown	0

Algorithm 4

Statistical Analysis

Step 1

Select numerical column

Step 2

Calculate Mean

Step 3

Calculate Median

Step 4

Calculate Mode

Step 5

Calculate Standard Deviation

Program 7

```
In [12]: print("Mean =", df["age"].mean())
         print("Median =", df["age"].median())
         print("Mode =")
         print(df["age"].mode())
         print("Standard Deviation =", df["age"].std())
```

```
Mean = 43.226614481409015
Median = 45.0
Mode =
0    78.0
Name: age, dtype: float64
Standard Deviation = 22.61264672311352
```

Important Formula

Mean

$$\bar{x} = \frac{\sum x}{n}$$

Program 8

Descriptive Statistics

```
In [13]: df.describe(include="all")
```

Out[13]:

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_st
count	5110.000000	5110	5110.000000	5110.000000	5110.000000	5110	5110	5110	5110.000000	4909.000000	
unique	NaN	3	NaN	NaN	NaN	2	5	2	NaN	NaN	
top	NaN	Female	NaN	NaN	NaN	Yes	Private	Urban	NaN	NaN	never sm
freq	NaN	2994	NaN	NaN	NaN	3353	2925	2596	NaN	NaN	
mean	36517.829354	NaN	43.226614	0.097456	0.054012	NaN	NaN	NaN	106.147677	28.893237	
std	21161.721625	NaN	22.612647	0.296607	0.226063	NaN	NaN	NaN	45.283560	7.854067	
min	67.000000	NaN	0.080000	0.000000	0.000000	NaN	NaN	NaN	55.120000	10.300000	
25%	17741.250000	NaN	25.000000	0.000000	0.000000	NaN	NaN	NaN	77.245000	23.500000	
50%	36932.000000	NaN	45.000000	0.000000	0.000000	NaN	NaN	NaN	91.885000	28.100000	
75%	54682.000000	NaN	61.000000	0.000000	0.000000	NaN	NaN	NaN	114.090000	33.100000	
max	72940.000000	NaN	82.000000	1.000000	1.000000	NaN	NaN	NaN	271.740000	97.600000	

Algorithm 5

Grouping and Aggregation

Step 1

Select Department Column

Step 2

Apply groupby()

Step 3

Calculate Average Marks

Step 4

Display Result

Program 9

```
In [14]: gender_avg = df.groupby("gender")["age"].mean()
         print(gender_avg)

gender
Female    43.757395
Male      42.483385
Other     26.000000
Name: age, dtype: float64
```

```
In [15]: work_avg = df.groupby("work_type")["bmi"].mean()
         print(work_avg)

work_type
Govt_job          30.522063
Never_worked      25.545455
Private           30.304625
Self-employed     30.211871
children          20.038003
Name: bmi, dtype: float64
```

Algorithm 6

Correlation Analysis

Step 1

Select numerical columns

Step 2

Apply corr()

Step 3

Interpret correlation coefficient

Program 10

```
In [16]: corr = df[["age", "avg_glucose_level", "bmi", "stroke"]].corr()
         print(corr)

                        age  avg_glucose_level       bmi    stroke
age                1.000000           0.238171  0.333398  0.245257
avg_glucose_level  0.238171           1.000000  0.175502  0.131945
bmi                0.333398           0.175502  1.000000  0.042374
stroke             0.245257           0.131945  0.042374  1.000000
```

Interpretation

Correlation = 0.95

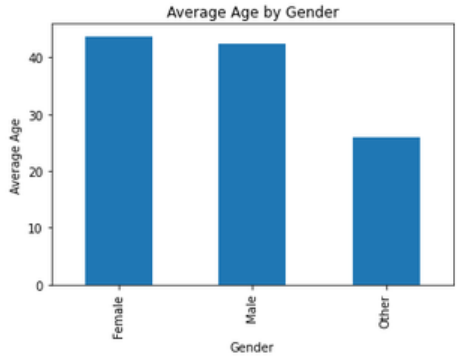
This indicates:

✓ Strong Positive Correlation

Visualization:

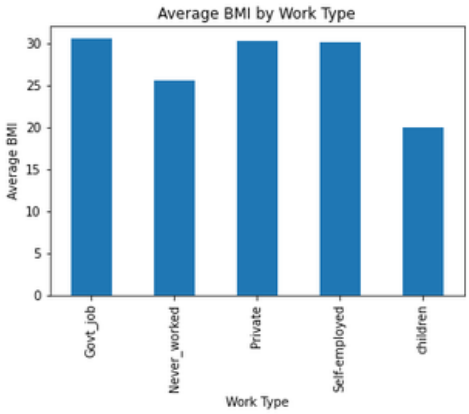
Age average Vs Gender :

```
7]: gender_avg.plot(kind="bar")

    plt.title("Average Age by Gender")
    plt.xlabel("Gender")
    plt.ylabel("Average Age")
    plt.show()
```



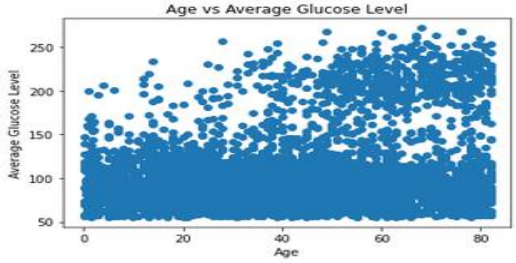
BMI average Vs Worktype:

```
In [18]: work_avg.plot(kind="bar")

         plt.title("Average BMI by Work Type")
         plt.xlabel("Work Type")
         plt.ylabel("Average BMI")
         plt.show()
```



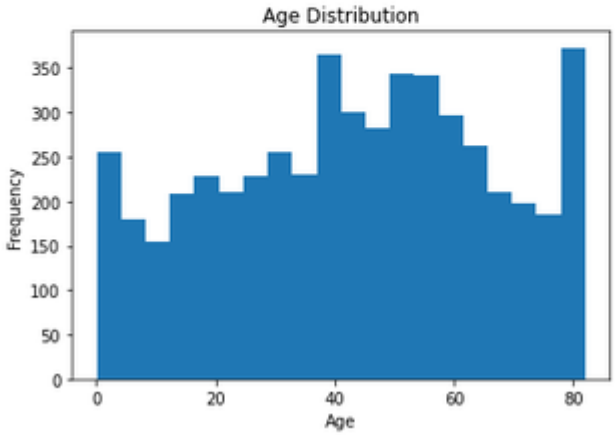
Scatter Plot of Age Vs Average Glucose Level

```
In [19]: plt.scatter(df["age"], df["avg_glucose_level"])

         plt.title("Age vs Average Glucose Level")
         plt.xlabel("Age")
         plt.ylabel("Average Glucose Level")
         plt.show()
```



Histogram of Age Distribution:

```
In [20]: plt.hist(df["age"], bins=20)

         plt.title("Age Distribution")
         plt.xlabel("Age")
         plt.ylabel("Frequency")
         plt.show()
```



Result

Thus, data inspection, filtering, sorting, descriptive statistical analysis, grouping, aggregation, and correlation analysis were successfully performed using Pandas. The analytical results were visualized and interpreted successfully.